

C023115

Software Testing- Unit 3

Black-Box Software
Testing (12M)



Black Box Testing

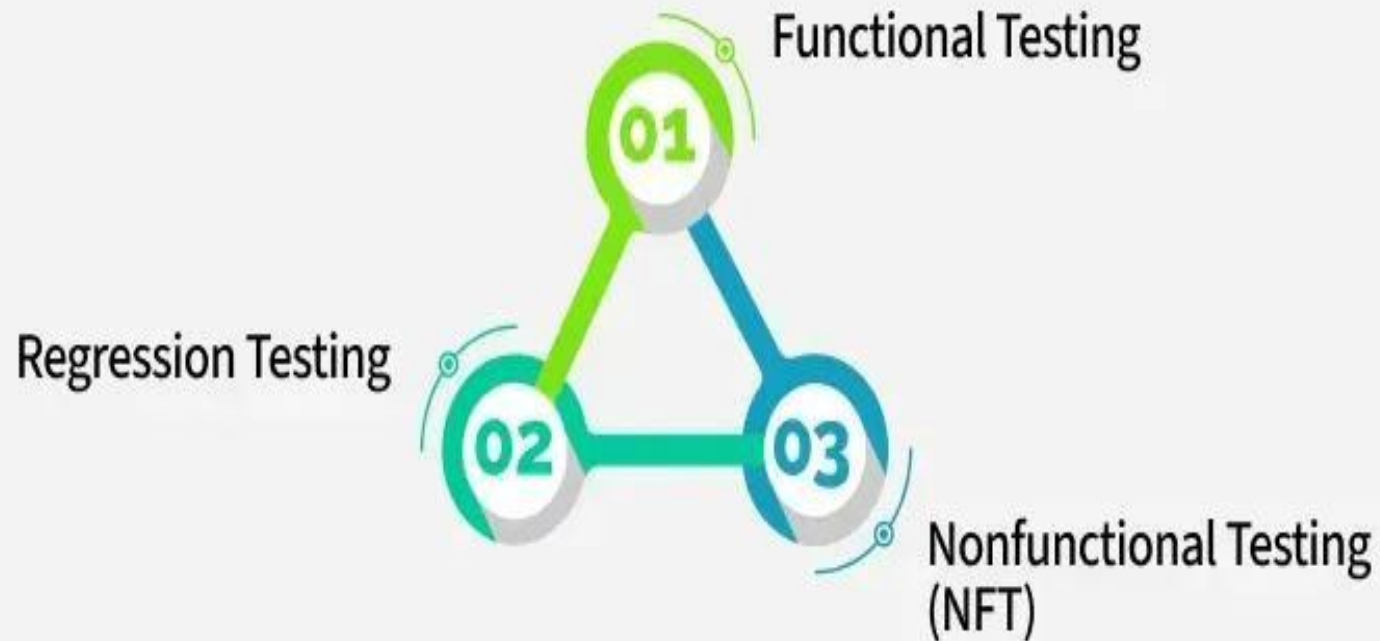
- Black-box testing is a Type of Software Testing in which the tester is not concerned with the software's internal knowledge or implementation details but rather focuses on validating the functionality based on the provided specifications or requirements.

Black Box Testing



Classification of BBT

Types Of Black Box Testing

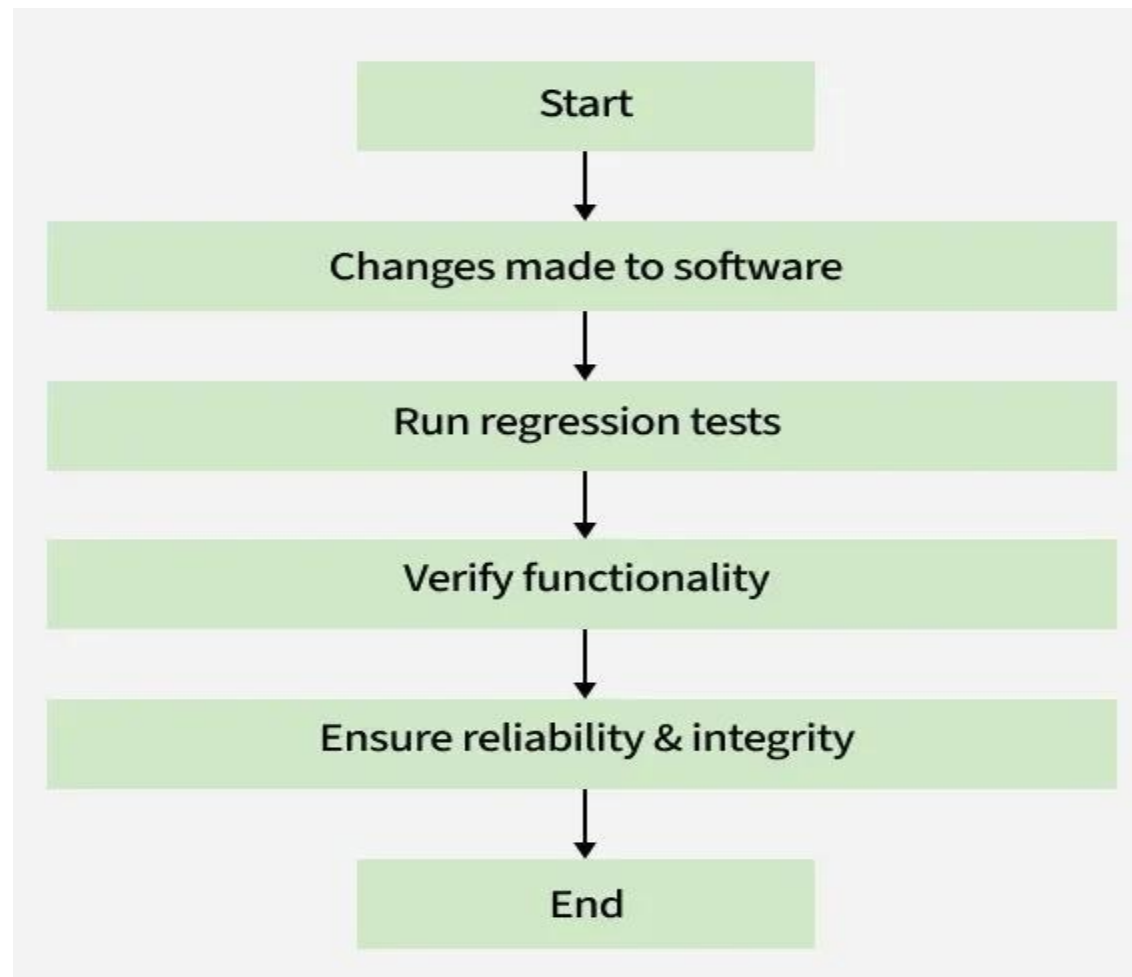


1. Functional Testing

- Functional Testing is a type of Software Testing in which the system is tested against the functional requirements and specifications.
- Functional testing ensures that the requirements or specifications are properly satisfied by the application.
- This testing is not concerned with the source code of the application.
- Each functionality of the software application is tested by providing appropriate test input, expecting the output, and comparing the actual output with the expected output.
- This testing focuses on checking the user interface, APIs, database, security, client or server application, and functionality of the Application Under Test. Functional testing can be manual or automated.
- It determines the system's software functional requirements.

2. Regression Testing

- **Regression Testing** involves re-executing a previously created test suite to verify that recent code changes haven't caused new issues. This verifies that updates, bug fixes, or enhancements do not break the functionality of the application.



When to do Regression Testing?

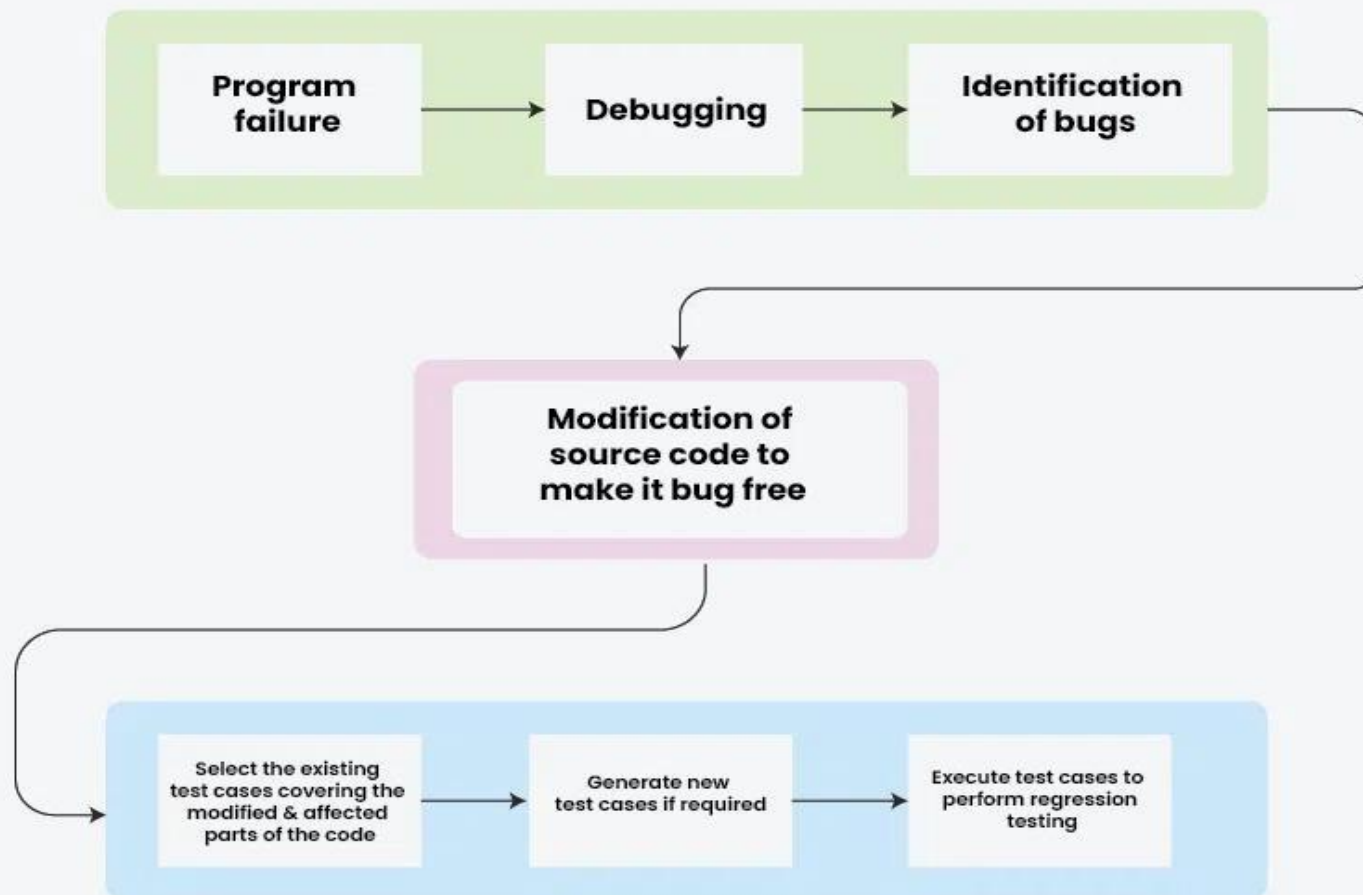
- Regression testing is necessary in several scenarios to maintain software quality:
 1. When new functionality is added to the system and the code has been modified to absorb and integrate that functionality with the existing code.
 2. When some defect has been identified in the software and the code is debugged to fix it.
 3. When the code is modified to optimize its working.

Process of Regression Testing

Here is the step-by-step process of the regression testing:



Process Regression testing



Non-Functional Testing

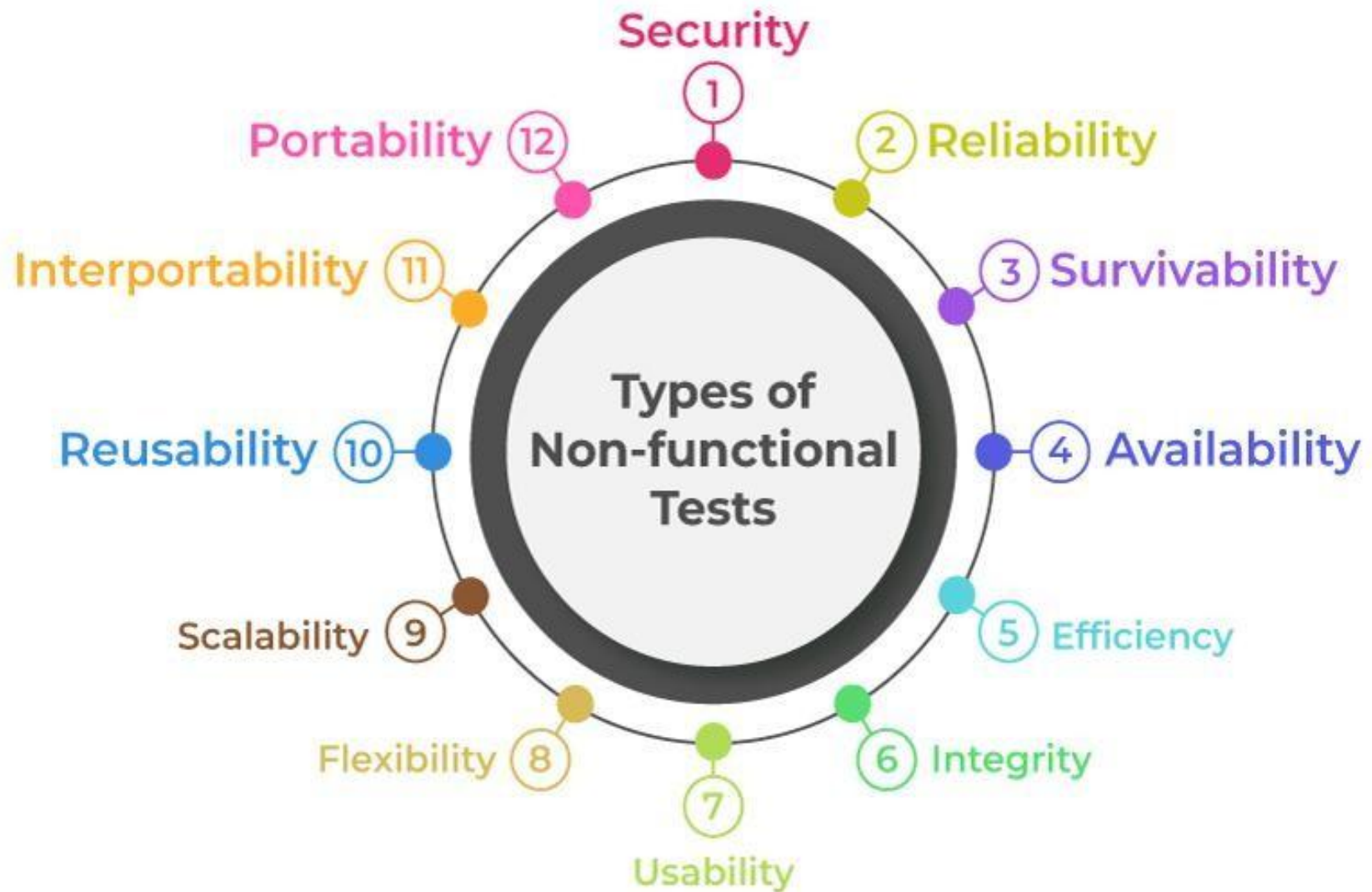
- **Non-functional Testing** is a type of software testing that is performed to verify the non-functional requirements of the application. It verifies whether the behavior of the system is as per the requirement or not.

❖ Non-Functional Testing Types and Techniques

Here is the Non-Functional Testing Techniques and Types mentioned below:

- **Performance Testing**: Performance testing is a type of software testing that focuses on evaluating the performance and scalability of a system or application.
- **Load Testing**: Load testing determines the behavior of the application when multiple users use it at the same time. It is the response of the system measured under varying load conditions.
- **Security Testing**: Security testing is important to check and sure that applications and systems are protected from various threats. There are several types of security testing, each targeting specific vulnerabilities and aspects of security.

Non-functional Testing Parameters



Black Box Testing Techniques

- Equivalence Partitioning
- Boundary Value Analysis
- Requirement-based testing
- Positive and Negative Testing

Equivalence Partitioning

- Testers can divide possible inputs into groups or “partitions”, and test only one example input from each group.
- For example, if a system requires a user’s birth date and provides the same response for all users under the age of 18, and a different response for users over 18, it is sufficient for testers to check one birth date in the “under 18” group and one date in the “over 18” group.



Equivalence Partitioning

- Instead of testing every possible input, a representative value from each partition is selected for testing, significantly reducing the number of test cases while maintaining high test coverage.

Black Box Testing Techniques

1. Equivalence Class Testing



Equivalence Partitioning

Example: Age Input Field

Consider a web application with an age input field that requires users to be between 18 and 60 years old (inclusive).

Identify Equivalence Classes:

Valid Equivalence Class: Ages between 18 and 60 (e.g., 25, 40, 55). Any age within this range should be accepted by the system.

Invalid Equivalence Class 1 (Below Range): Ages less than 18 (e.g., 17, 10, 0). These should be rejected.

Invalid Equivalence Class 2 (Above Range): Ages greater than 60 (e.g., 61, 70, 100). These should be rejected.

Invalid Equivalence Class 3 (Non-numeric): Inputs that are not numbers (e.g., "abc", "!@#", "twenty"). These should be rejected.



Boundary Value Analysis (BVA)

- Boundary Value Analysis (BVA) is a crucial software testing technique that focuses on testing the boundaries or edges of input ranges.
- It is based on the observation that errors often occur at the edges of input ranges rather than in the middle. By testing the extremes, BVA helps identify potential vulnerabilities and ensures that the system behaves correctly at its limits.

Black Box Testing Techniques

2. Boundary Value Analysis



Boundary Value Analysis (BVA)

- Boundary Value Analysis is a black-box testing method that aims to test the boundaries between equivalence classes.
- While Equivalence Class Partitioning only divides inputs into valid and invalid groups, BVA takes this a step further by testing only extreme values *just below*, *just above*, and *at* the defined boundaries.
- This helps ensure the system properly handles edge cases, which are often where software failures occur.



Boundary Value Analysis (BVA)

Let's go back to the previous example. You're testing a function that validates a user's age, with a valid age range of 18 to 60 years. We have the following equivalence classes already defined:

Valid Equivalence Class: [18–60]

Invalid Equivalence Classes:

Ages less than 18: [$-\infty$ to 17]

Ages greater than 60: [61 to ∞]

Non-numeric inputs: ["abc", "#\$%", etc.]

Boundary value analysis focuses on testing the boundaries of these ranges.

Just outside the lower boundary: 17

At the lower boundary: 18

Just above the lower boundary: 19

Just below the upper boundary: 59

At the upper boundary: 60

Just outside the upper boundary: 61



Requirement-based testing

Let's go back to the previous example. You're testing a function that validates a user's age, with a valid age range of 18 to 60 years. We have the following equivalence classes already defined:

Valid Equivalence Class: [18–60]

Invalid Equivalence Classes:

Ages less than 18: [$-\infty$ to 17]

Ages greater than 60: [61 to ∞]

Non-numeric inputs: ["abc", "\$%", etc.]

Boundary value analysis focuses on testing the boundaries of these ranges.

Just outside the lower boundary: 17

At the lower boundary: 18

Just above the lower boundary: 19

Just below the upper boundary: 59

At the upper boundary: 60

Just outside the upper boundary: 61



Requirement-based testing

Let's go back to the previous example. You're testing a function that validates a user's age, with a valid age range of 18 to 60 years. We have the following equivalence classes already defined:

Valid Equivalence Class: [18–60]

Invalid Equivalence Classes:

Ages less than 18: [$-\infty$ to 17]

Ages greater than 60: [61 to ∞]

Non-numeric inputs: ["abc", "\$%", etc.]

Boundary value analysis focuses on testing the boundaries of these ranges.

Just outside the lower boundary: 17

At the lower boundary: 18

Just above the lower boundary: 19

Just below the upper boundary: 59

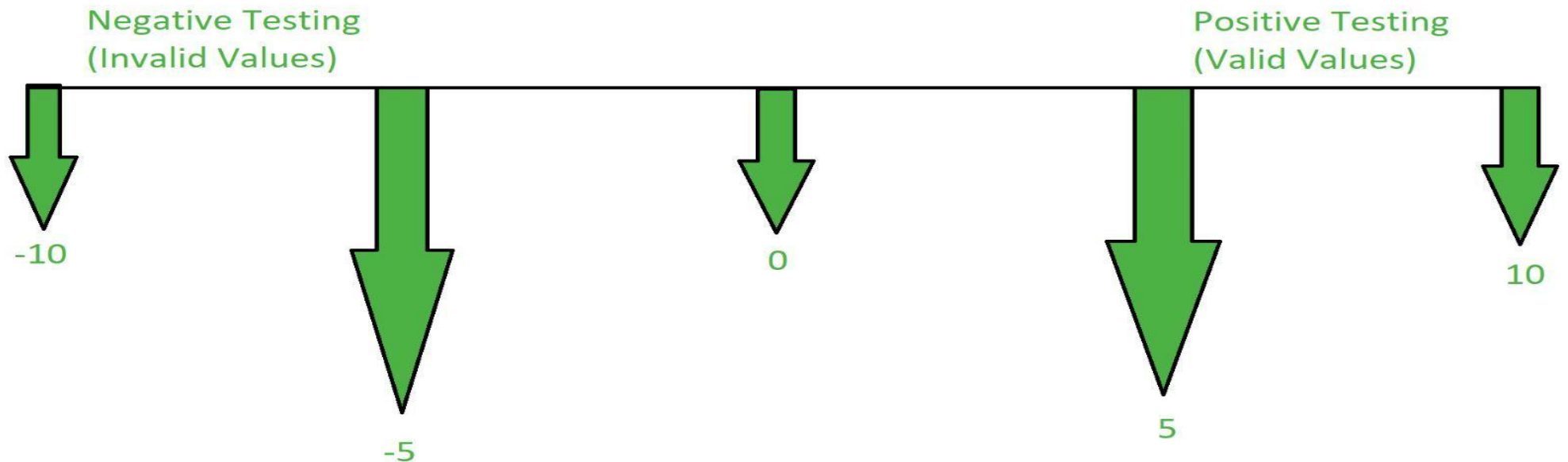
At the upper boundary: 60

Just outside the upper boundary: 61



Positive and Negative Testing

- **Positive testing:** is a type of software testing that is performed by assuming everything will be as expected.
- It is performed with the assumption that only valid and relevant things will occur.
- Data set and all other functionalities will be as expected.
- **Negative Testing:** Negative testing is a type of software testing that is performed to check the system for unexpected conditions.
- Negative testing plays a much important role in high-performance software development.
- It checks whether on such unexpected conditions what will be the behavior of the software.



Positive test cases vs. negative test cases

Positive test cases	Negative test cases
How a system reacts to valid input data	How a system reacts to invalid input data
How software handles expected conditions	How software handles unexpected conditions
How the app behaves against software requirements	How the app behaves beyond the scope of software requirements
	

Gray Box Testing

- **Gray Box Testing** is a software testing technique that combines elements of the Black Box Testing technique and the White Box Testing technique.

Objectives of Gray Box Testing:

- To provide the combined advantages of both black box testing and white box testing.
- To combine the input of developers as well as testers.
- To improve overall product quality.
- To reduce the overhead of the long process of functional and non-functional testing.
- To provide enough free time to developers to fix defects.
- To test from the user's point of view rather than a designer's point of view.



Gray Box Testing Techniques



Gray Box Testing Techniques

